Sveučilište Josipa Jurja Strossmayera u Osijeku
Građevinski i arhitektonski fakultet Osijek
Josip Juraj Strossmayer University of Osijek
Faculty of Civil Engineering and Architecture Osijek

Računalno programiranje u građevinarstvu (254810) | Tjedan 2/15
Akademska godina 2025./2026.

Strukture podataka i logika

Liste, n-torke, rječnici, logički operatori, grananja i petlje

Marin Grubišić
Izv. prof. dr. sc. / mag. ing. aedif.

Sveučilište J. J. Strossmayera u Osijeku (UNIOS)
Građevinski i arhitektonski fakultet Osijek (GrAFOS)
Zavod za tehničku mehaniku (ZTM)
Predsjednik Katedre za teoriju konstrukcija
Vladimira Preloga 3, **Ured II.26**, HR–31 000 Osijek, Hrvatska

marin.grubisic@gfos.hr | **www.maringrubisic.com**

Structural &
Earthquake Engineering

Sadržaj prezentacije

1 GenAI

2 Strukture podataka

3 Logički operatori i grananje

4 Petlje

5 Praktični primjeri

6 Zaključci

Alati za generativnu umjetnu inteligenciju (GenAI)

- Studentima je dozvoljeno služiti se **generativnom umjetnom inteligencijom (GenAI)** za potrebe generiranje programskog kôda u Python programskom jeziku (ChatGPT, Gemini, Grok, DeepSeek, Kimi, ...).

Alati za **generativnu umjetnu inteligenciju (GenAI)**

- OpenAI ChatGPT (**chatgpt.com**).
- Google Gemini (**gemini.google.com**).
- xAI Grok (**grok.com**).
- DeepSeek AI (**deepseek.com**).
- Moonshot Kimi (**kimi.com**).

Pregled struktura podataka u Pythonu

Strukture podataka — uvod i pregled

U Pythonu postoji nekoliko temeljnih struktura za pohranu skupa podataka. Svaka ima svoju primjenu i karakteristike:

Struktura	Sintaksa	Karakteristike	Promjenjiva?
Lista	`[a, b, c]`	Uređen niz, bilo koji tipovi	Da
N-torka	`(a, b, c)`	Uređen niz, nepromjenjiva	Ne
Rječnik	`{k: v}`	Parovi ključ–vrijednost	Da
Skup	`{a, b, c}`	Neuređen, bez duplikata	Da

U ovom kolegiju naglasak je na **listama** i **rječnicima** — najkorisnije strukture za inženjerske proračune i organizaciju podataka.

Liste — kreiranje i indeksiranje

Strukture podataka — lista (`list`)

Lista (`list`) je uređeni niz elemenata koji mogu biti bilo kojeg tipa:

Algoritam 2.1

```
# Kreiranje lista
rasponi = [4.0, 5.5, 6.0, 5.5, 4.0]     # [m] rasponi greda
sile    = [10.5, 22.0, 15.3, 18.7]       # [kN] sile u čvorovima
materijali = ["beton", "čelik", "drvo"]  # lista stringova
mijesano   = [30.0, "C30/37", True, 4]   # mješoviti tipovi

# Indeksiranje --- POČINJE OD 0!
print(rasponi[0])    # 4.0   (prvi element)
print(rasponi[2])    # 6.0   (treći element)
print(rasponi[-1])   # 4.0   (zadnji element)
print(rasponi[-2])   # 5.5   (predzadnji)

# Duljina liste
print(len(rasponi))  # 5
```

> Indeksi u Pythonu počinju od **0**, ne od 1! Negativni indeksi broje od kraja liste.

Liste — rezanje (slicing) i metode

Strukture podataka — lista (`list`)

Algoritam 2.2

```
rasponi = [4.0, 5.5, 6.0, 5.5, 4.0]  # [m]

# Rezanje: lista[start:stop]  (stop nije uključen!)
print(rasponi[1:3])   # [5.5, 6.0]
print(rasponi[:2])    # [4.0, 5.5]  (od početka do indeksa 2)
print(rasponi[2:])    # [6.0, 5.5, 4.0] (od indeksa 2 do kraja)

# Ugrađene metode lista
rasponi.append(7.0)   # dodaj element na kraj
rasponi.insert(2, 3.0)  # umetni na indeks 2
rasponi.pop()         # ukloni i vrati zadnji element
rasponi.sort()        # sortiraj uzlazno
rasponi.reverse()     # obrnuti redoslijed

# Korisne funkcije
print(max(rasponi))   # najveći element
print(min(rasponi))   # najmanji element
print(sum(rasponi))   # zbroj svih elemenata
```

N-torke (tuple)

Strukture podataka — n-torka (`tuple`)

N-torka (`tuple`) je uređeni niz poput liste, ali je **nepromjenjiva** (immutable) — vrijednosti se ne mogu mijenjati nakon kreiranja:

Algoritam 2.3

```
# Kreiranje n-torki
presjek = (0.30, 0.50)      # (b, h) dimenzije u [m]
cvor_A  = (0.0, 0.0, 0.0)   # (x, y, z) koordinate čvora

# Indeksiranje (isto kao kod lista)
b = presjek[0]  # 0.30 [m]
h = presjek[1]  # 0.50 [m]

# Raspakiranje (unpacking)
b, h = presjek         # elegantno dohvaćanje vrijednosti
x, y, z = cvor_A

# N-torke se NE mogu mijenjati --- ovo bi izazvalo grešku:
# presjek[0] = 0.40   # TypeError!

# Provjera tipa
print(type(presjek))  # <class 'tuple'>
```

> N-torke su idealne za **fiksne inženjerske konstante** (koordinate čvorova, dimenzije presjeka) koje se ne smiju mijenjati tijekom proračuna.

Rječnici — kreiranje i pristup

Strukture podataka — rječnik (`dict`)

Rječnik (`dict`) pohranjuje podatke kao parove **ključ: vrijednost** — savršen za organizaciju inženjerskih podataka:

Algoritam 2.4

```
# Kreiranje rječnika --- opis materijala betona C30/37
beton = {
    "naziv"   : "C30/37",
    "fck"     : 30.0,     # [MPa] karakteristična tlačna čvrstoća
    "fcd"     : 20.0,     # [MPa] projektna tlačna čvrstoća
    "Ecm"     : 32836.6,  # [MPa] sekantni modul elastičnosti
    "gamma_c" : 1.5,      # [-]   parcijalni faktor sigurnosti
    "armiran" : False     # [-]   nearmiran
}

# Pristup vrijednostima
print(beton["fck"])       # 30.0
print(beton["naziv"])     # C30/37

# Dodavanje novog ključa
beton["nu"] = 0.2         # Poissonov koeficijent

# Provjera postoji li ključ
print("fcd" in beton)     # True
```

Rječnici — metode i iteracija

Strukture podataka — rječnik (`dict`)

Algoritam 2.5

```python
beton = {"naziv": "C30/37", "fck": 30.0, "Ecm": 32836.6}

# Dohvat svih ključeva i vrijednosti
print(beton.keys())    # dict_keys(['naziv', 'fck', 'Ecm'])
print(beton.values())  # dict_values(['C30/37', 30.0, 32836.6])

# Iteracija po rječniku
for kljuc, vrijednost in beton.items():
    print(kljuc, "=", vrijednost)

# Sigurno dohvaćanje (bez greške ako ključ ne postoji)
nu = beton.get("nu", 0.2)   # vraća 0.2 ako "nu" ne postoji

# Lista rječnika --- baza materijala
materijali = [
    {"naziv": "C25/30", "fck": 25.0, "Ecm": 31000.0},
    {"naziv": "C30/37", "fck": 30.0, "Ecm": 32837.0},
    {"naziv": "C35/45", "fck": 35.0, "Ecm": 34077.0},
]
print(materijali[1]["fck"])  # 30.0
```

Operatori usporedbe

Logički operatori i grananje — operatori usporedbe

Operatori usporedbe vraćaju `True` ili `False`:

Algoritam 3.1

```
sigma_Ed = 18.5   # [MPa] projektno naprezanje
fcd      = 20.0   # [MPa] projektna čvrstoća

print(sigma_Ed <  fcd)   # True  (manje od)
print(sigma_Ed >  fcd)   # False (veće od)
print(sigma_Ed <= fcd)   # True  (manje ili jednako)
print(sigma_Ed >= fcd)   # False (veće ili jednako)
print(sigma_Ed == fcd)   # False (jednako)  --- dva znaka jednakosti!
print(sigma_Ed ≠ fcd)   # True  (različito)

# Česta greška: = je dodjela, == je usporedba!
# sigma_Ed = fcd   # mijenja vrijednost varijable!
# sigma_Ed == fcd  # uspoređuje vrijednosti
```

Operator **dodjele** je `=`, a operator **usporedbe** je `==`. Miješanje ova dva operatora je česta početna greška!

Logički operatori

Logički operatori i grananje — `and`, `or`, `not`

Logički operatori `and`, `or` i `not` omogućuju kombiniranje uvjeta:

Algoritam 3.2

```
sigma_Ed = 18.5   # [MPa] projektno naprezanje od savijanja
tau_Ed   = 1.2    # [MPa] projektno posmično naprezanje
fcd      = 20.0   # [MPa] projektna tlačna čvrstoća
fctd     = 1.8    # [MPa] projektna vlačna čvrstoća

# AND: oba uvjeta moraju biti ispunjena
nosivost_ok = (sigma_Ed <= fcd) and (tau_Ed <= fctd)
print(nosivost_ok)   # True

# OR: barem jedan uvjet mora biti ispunjen
potrebna_provjera = (sigma_Ed > 0.8*fcd) or (tau_Ed > 0.8*fctd)
print(potrebna_provjera)  # True

# NOT: negacija
nije_ok = not (sigma_Ed < fcd)
print(nije_ok)   # False

# Kombinirani uvjeti
print((sigma_Ed > 0) and (sigma_Ed <= fcd) and (tau_Ed <= fctd))
```

Uvjetna naredba `if` / `elif` / `else`

Logički operatori i grananje — grananja

Algoritam 3.3

```python
# Provjera nosivosti presjeka
sigma_Ed = 18.5   # [MPa] projektno naprezanje
fcd      = 20.0   # [MPa] projektna čvrstoća betona

if sigma_Ed <= 0.6 * fcd:
    print("Naprezanje u normalnom rasponu.")
elif sigma_Ed <= fcd:
    print("Naprezanje povišeno, ali unutar dozvoljenih granica.")
elif sigma_Ed <= 1.1 * fcd:
    print("UPOZORENJE: Naprezanje prekoračuje projektnu čvrstoću!")
else:
    print("GREŠKA: Presjek nije nosiv! Potrebno redimenzioniranje.")

# Jednolinijsko grananje (ternary operator)
status = "OK" if sigma_Ed <= fcd else "NOK"
print("Status:", status)
```

> Python koristi **uvlačenje (indentacija)** od **4 razmaka** umjesto vitičastih zagrada za definiranje blokova koda. Konzistentna indentacija je obavezna!

Ugniježđena grananja i složeni uvjeti

Logički operatori i grananje — ugniježđena grananja

Algoritam 3.4

```python
# Klasifikacija AB presjeka prema razini iskorištenosti
M_Ed  = 150.0   # [kNm] projektni moment savijanja
b     = 0.30    # [m]
h     = 0.50    # [m]
fck   = 30.0    # [MPa]
fcd   = fck / 1.5

Wy = (b * h**2) / 6 # [m³] moment otpora
sigma_Ed = (M_Ed / Wy) / 1000  # [MPa] (M u kNm -> 1000 za MPa)

iskorištenost = sigma_Ed / fcd  # omjer iskorištenosti

if iskorištenost <= 1.0:
    if iskorištenost <= 0.6:
        razina = "nisko iskorišten"
    elif iskorištenost <= 0.85:
        razina = "umjereno iskorišten"
    else:
        razina = "visoko iskorišten"
    print(f"Presjek je {razina} ({iskorištenost:.1%})")
else:
    print(f"Presjek nije nosiv! (iskorištenost: {iskorištenost:.1%})")
```

Petlja `for` — osnove

Petlje — iteracija s `for`

Petlja `for` iterira po elementima neke sekvence (liste, n-torke, stringa...):

Algoritam 4.1

```
# Iteracija po listi raspona greda
rasponi = [4.0, 5.5, 6.0, 5.5, 4.0]   # [m]

for L in rasponi:
    print("Raspon:", L, "m")

# Izračun za svaki element liste
q = 15.0   # [kN/m] jednoliko opterećenje
print("\nProračun momenata:")
for L in rasponi:
    M_max = q * L**2 / 8    # [kNm] max. moment proste grede
    print(f"  L = {L:.1f} m  -->  M_max = {M_max:.2f} kNm")
```

f-stringovi (`f"..."`) omogućuju elegantno formatiranje ispisa:

- `f"{L:.1f}"` — decimalni broj s 1 decimalnim mjestom
- `f"{M_max:.2f}"` — decimalni broj s 2 decimalna mjesta
- `f"{iskor:.1%}"` — postotak s 1 decimalnim mjestom

Formatiranje ispisa — f-stringovi

Petlje — formatiranje rezultata s f-stringovima

F-string (`f"..."`) je moderan i preporučeni način formatiranja ispisa u Pythonu. Varijabla ili izraz ugrađuje se direktno u string unutar vitičastih zagrada `{...}` :

Algoritam 4.2

```python
b, h  = 0.30, 0.55  # [m]
M_Ed  = 187.24      # [kNm]
sigma = 14.567      # [MPa]
ok    = True

# Bez formatiranja --- nečitljiv ispis
print("sigma =", sigma, "MPa")      # sigma = 14.567 MPa
# F-string --- osnovna ugradnja varijable
print(f"sigma = {sigma} MPa")       # sigma = 14.567 MPa

# Decimalna mjesta: {varijabla:.Nf}
print(f"sigma = {sigma:.2f} MPa")   # sigma = 14.57 MPa
print(f"M_Ed  = {M_Ed:.1f} kNm")    # M_Ed  = 187.2 kNm
# Širina polja (poravnanje): {varijabla:>Nf} ili {:>Ns}
print(f"{'sigma':>10} = {sigma:>8.3f} MPa")   # desno poravnanje
print(f"{'M_Ed':>10} = {M_Ed:>8.2f} kNm")

# Postotak: {varijabla:.N%}
iskor = sigma / 20.0
print(f"Iskorištenost: {iskor:.1%}")   # Iskorištenost: 72.8%
# Izraz unutar vitičastih zagrada
print(f"Wy = {(b * h**2 / 6) * 1e6:.1f} cm³")  # Wy = 1507.5 cm³
```

Formatiranje ispisa — f-stringovi

Petlje — formatiranje rezultata s f-stringovima

F-string (`f"..."`) je moderan i preporučeni način formatiranja ispisa u Pythonu (od verzije 3.6). Varijabla ili izraz ugrađuje se direktno u string unutar vitičastih zagrada `{...}` :

Najčešće korišteni specifikatori formata:

- `:.2f` (2 decimale),
- `:.3e` (znanstvena notacija),
- `:.1%` (postotak),
- `:>10.2f` (desno poravnanje u polju širine 10).

Petlja `for` — `range()` i `enumerate()`

Petlje — `range()` i `enumerate()`

Algoritam 4.3

```python
# range(start, stop, korak) --- generira niz cijelih brojeva
for i in range(5):  # 0, 1, 2, 3, 4
    print(i, end=" ")

for i in range(1, 6):  # 1, 2, 3, 4, 5
    print(i, end=" ")

# Korisna primjena: indeksiranje i brojanje
rasponi = [4.0, 5.5, 6.0, 5.5, 4.0]
for i in range(len(rasponi)):
    print(f"Greda {i+1}: L = {rasponi[i]:.1f} m")

# enumerate() --- elegantno dohvaćanje indeksa i vrijednosti
print("\nKoristeći enumerate:")
for i, L in enumerate(rasponi, start=1):
    print(f"Greda {i}: L = {L:.1f} m")
```

> `enumerate(lista, start=1)` vraća parove `(indeks, vrijednost)` i preporuča se umjesto `range(len(lista))` zbog čitljivosti koda.

Petlja `while`

Petlje — iteracija s `while`

Petlja `while` izvršava blok koda **sve dok je uvjet ispunjen**:

Algoritam 4.4

```
# Iterativni proračun: povećavanje presjeka dok nije nosiv
b      = 0.20    # [m] početna širina presjeka
h      = 0.30    # [m] početna visina presjeka
M_Ed   = 120.0   # [kNm] projektni moment
fcd    = 20.0    # [MPa] projektna čvrstoća

iteracija = 0
while True:
    Wy = (b * h**2) / 6          # [m³]
    sigma = (M_Ed / Wy) / 1000   # [MPa]
    iteracija += 1

    if sigma <= fcd:
        print(f"Rješenje nađeno u iteraciji {iteracija}:")
        print(f"  b = {b:.2f} m, h = {h:.2f} m")
        print(f"  sigma = {sigma:.2f} MPa ≤ fcd = {fcd:.1f} MPa")
        break   # izlaz iz petlje
    else:
        h += 0.05   # povećaj visinu za 5 cm
```

Kontrola petlji: `break` i `continue`

Petlje — `break` i `continue`

Algoritam 4.5

```python
sile = [12.5, -3.2, 8.7, -15.0, 6.1, 22.3, -1.5]  # [kN]

# break --- prekida petlju kad je uvjet ispunjen
print("Tražim prvu silu > 20 kN:")
for i, F in enumerate(sile):
    if F > 20.0:
        print(f"  Nađena: F[{i}] = {F:.1f} kN")
        break   # izlaz iz petlje čim nađemo

# continue --- preskače trenutnu iteraciju
print("\nSamo pozitivne sile:")
for F in sile:
    if F <= 0:
        continue   # preskoči negativne
    print(f"  F = {F:.1f} kN")
```

- `break` — trenutni izlaz iz petlje
- `continue` — skok na sljedeću iteraciju
- Oba se mogu koristiti i u `for` i u `while` petljama

Primjer 1: Provjera nosivosti skupa greda

Praktični primjeri — nosive konstrukcije

Algoritam 5.1

```python
# Provjera naprezanja u skupu kontinuiranih greda
import math

grede = [
    {"oznaka": "G1", "L": 5.0, "b": 0.25, "h": 0.45},
    {"oznaka": "G2", "L": 6.5, "b": 0.30, "h": 0.55},
    {"oznaka": "G3", "L": 4.0, "b": 0.25, "h": 0.40},
]
q   = 20.0   # [kN/m] jednoliko opterećenje
fcd = 20.0   # [MPa] projektna čvrstoća betona

print(f"{'Greda':>5} | {'L[m]':>6} | {'M_Ed[kNm]':>10} | "
      f"{'sigma[MPa]':>10} | {'Status':>8}")
print("-" * 55)
for g in grede:
    M_Ed  = q * g["L"]**2 / 8
    Wy    = (g["b"] * g["h"]**2) / 6
    sigma = (M_Ed / Wy) / 1000
    ok    = "OK" if sigma <= fcd else "NOK"
    print(f"{g['oznaka']:>5} | {g['L']:>6.1f} | {M_Ed:>10.2f} | "
          f"{sigma:>10.2f} | {ok:>8}")
```

Primjer 2: Proračun ukupne mase konstrukcije

Praktični primjeri — nosive konstrukcije

Algoritam 5.2

```python
# Masa i težina AB elemenata okvira
# Dimenzije: (b [m], h [m], L [m])
elementi = {
    "Stup S1" : (0.40, 0.40, 3.50),
    "Stup S2" : (0.40, 0.40, 3.50),
    "Stup S3" : (0.50, 0.50, 3.50),
    "Greda G1": (0.30, 0.55, 6.00),
    "Greda G2": (0.30, 0.55, 5.50),
    "Greda G3": (0.25, 0.45, 4.00),
}
gamma_beton = 25.0   # [kN/m³] specifična težina betona

ukupna_V = 0.0
ukupna_G = 0.0
print(f"{'Element':>10} | {'V [m³]':>8} | {'G [kN]':>8}")
print("-" * 34)
for naziv, (b, h, L) in elementi.items():
    V = b * h * L
    G = gamma_beton * V
    ukupna_V += V
    ukupna_G += G
    print(f"{naziv:>10} | {V:>8.3f} | {G:>8.2f}")
print("-" * 34)
print(f"{'UKUPNO':>10} | {ukupna_V:>8.3f} | {ukupna_G:>8.2f}")
```

Primjer 3: Geometrijska svojstva skupa presjeka

Praktični primjeri — nosive konstrukcije

Algoritam 5.3

```python
# Geometrijska svojstva pravokutnih presjeka
presjeci = [
    (0.20, 0.40),   # (b, h) u [m]
    (0.25, 0.50),
    (0.30, 0.60),
    (0.35, 0.70),
]

print(f"{'b[m]':>6} {'h[m]':>6} {'A[cm²]':>9} "
      f"{'I[cm⁴]':>12} {'W[cm³]':>10} {'i[cm]':>8}")
print("-" * 55)

for b, h in presjeci:
    A  = b * h
    I  = (b * h**3) / 12
    W  = (b * h**2) / 6
    i  = (I / A)**0.5   # polumjer tromosti

    # Pretvorba u praktičnije jedinice
    print(f"{b:>6.2f} {h:>6.2f} {A*1e4:>9.1f} "
          f"{I*1e8:>12.1f} {W*1e6:>10.1f} {i*1e2:>8.2f}")
```

Primjer 4: Iterativno traženje minimalnog presjeka

Praktični primjeri — nosive konstrukcije

Algoritam 5.4

```python
# Traženje minimalnog pravokutnog presjeka uz ograničenje naprezanja
# i progiba metodom iscrpnog pretraživanja (brute force)

M_Ed    = 200.0      # [kNm] projektni moment
q       = 30.0       # [kN/m] jednoliko opterećenje
L       = 7.0        # [m] raspon grede
fcd     = 20.0       # [MPa] projektna čvrstoća
Ecm     = 32000.0    # [MPa] -> [kN/m²] = Ecm * 1000
L_250   = L / 250    # [m] granični progib

omjer_bh = 2.0   # pretpostavka h = 2b

rješenja = []
for n in range(1, 21):       # b od 0.10 do 1.00 m (korak 5 cm)
    b = 0.10 + (n - 1) * 0.05
    h = omjer_bh * b
    I     = (b * h**3) / 12
    Wy    = (b * h**2) / 6
    sigma = (M_Ed / Wy) / 1000
    w_max = (5 * q * L**4) / (384 * (Ecm * 1000) * I)
    if sigma <= fcd and w_max <= L_250:
        rješenja.append((b, h, sigma, w_max * 1000))
        break   # prvi (najmanji) valjani presjek

if rješenja:
    b, h, sigma, w = rješenja[0]
    print(f"Minimalni presjek: b={b:.2f} m, h={h:.2f} m")
    print(f"sigma = {sigma:.2f} MPa,  w = {w:.1f} mm")
```

Primjer 5: Kombinacije opterećenja prema EN 1990

Praktični primjeri — nosive konstrukcije

Algoritam 5.5

```
# Osnovna kombinacija opterećenja za GNS prema EN 1990
# E_d = gamma_G * G_k + gamma_Q * Q_k + suma(psi_0i * Q_ki)

gamma_G = 1.35   # [-] parcijalni faktor za stalno opterećenje
gamma_Q = 1.50   # [-] parcijalni faktor za promjenjivo opterećenje

# Karakteristična opterećenja [kN/m]
G_k  = 12.0    # stalno opterećenje (vlastita težina + stalno)
Q_k1 = 8.0     # korisno opterećenje (kategorija B)
Q_k2 = 3.5     # opterećenje snijegom
psi_01 = 0.7   # faktor kombinacije za korisno opterećenje
psi_02 = 0.5   # faktor kombinacije za snijeg

# Kombinacija 1: Q_k1 dominantno promjenjivo opterećenje
E_d1 = gamma_G*G_k + gamma_Q*Q_k1 + psi_02*Q_k2
# Kombinacija 2: Q_k2 dominantno promjenjivo opterećenje
E_d2 = gamma_G*G_k + psi_01*Q_k1 + gamma_Q*Q_k2

mjerodavna = max(E_d1, E_d2)
print(f"Kombinacija 1: E_d1 = {E_d1:.2f} kN/m")
print(f"Kombinacija 2: E_d2 = {E_d2:.2f} kN/m")
print(f"Mjerodavna:    E_d  = {mjerodavna:.2f} kN/m")
```

Zaključci

Pregled najvažnijih aspekata ovog tjedna

Što smo naučili:

- **Strukture podataka:** lista (`list`), n-torka (`tuple`) i rječnik (`dict`) — organizacija inženjerskih podataka
- **Indeksiranje i rezanje** lista (indeksi od `0`, negativni indeksi od kraja)
- **Metode lista i rječnika:** `append()`, `sort()`, `get()`, `items()`, `keys()`
- **Operatori usporedbe** (`<`, `>`, `==`, `≠` ...) i **logički operatori** (`and`, `or`, `not`)
- **Grananja:** `if` / `elif` / `else` za uvjetno izvršavanje koda
- **Petlja** `for`**:** iteracija po listama, `range()`, `enumerate()`
- **Petlja** `while`**:** ponavljanje sve dok je uvjet ispunjen
- **Kontrola toka:** `break` i `continue`
- **f-stringovi** za elegantno formatiranje numeričkih rezultata

Sljedeći tjedan (3/15): NumPy — vektori, matrice i rješavanje sustava linearnih jednadžbi.

Domaća zadaća #2

Rok predaje: do početka sljedećeg termina nastave

Zadatak: Napišite Python program koji analizira **jednu armiranobetonsku gredu** s poznatim rasponom L, širinom presjeka b, visinom presjeka h i jednolikim opterećenjem q. Vrijednosti zadajte kao varijable u sklopu **riječnika** na početku programa.

Izračunajte sljedeće veličine:

1. Maksimalni moment savijanja: $M_{\text{max}} = \dfrac{q\,L^2}{8}$ [kNm]
2. Moment otpora presjeka: $W_y = \dfrac{b\,h^2}{6}$ [m^3]
3. Naprezanje od savijanja: $\sigma = \dfrac{M_{\text{max}}}{W_y \cdot 1000}$ [MPa]
4. Stupanj iskorištenosti: $\eta = \dfrac{\sigma}{f_{cd}} \cdot 100$ [%], $\quad f_{cd} = 20{,}0$ MPa

Domaća zadaća #2

Rok predaje: do početka sljedećeg termina nastave

Koristeći **isključivo grananja** (`if` / `elif` / `else`) odredite i ispišite:

- **Status nosivosti:** `"OK"` ako $\sigma \leq f_{cd}$, inače `"NOK"`
- **Razred iskorištenosti:** `"Nisko"` ($\eta \leq 50\,\%$), `"Srednje"` ($50 < \eta \leq 80\,\%$), `"Visoko"` ($80 < \eta \leq 100\,\%$), `"Prekoračenje!"` ($\eta > 100\,\%$)
- **Vitkost grede** na temelju omjera L/h: `"Kratka"` ($L/h \leq 10$), `"Standardna"` ($10 < L/h \leq 20$), `"Vitka - provjeriti progib!"` ($L/h > 20$)
- **Preporuka betona** na temelju η: `"C20/25 dovoljan"` ($\eta \leq 50\,\%$), `"C25/30 preporučen"` ($50 < \eta \leq 80\,\%$), `"C30/37 preporučen"` ($80 < \eta \leq 100\,\%$), `"Reprojektirati presjek!"` ($\eta > 100\,\%$)

Zahtjevi: ispis u urednoj formi s `print` naredbom; smisleni komentari u kôdu; **poželjno je koristiti rječnik za definiciju ulaznih parametara**. Nije potrebno korištenje petlji.

Hvala na pozornosti!

Pitanja?

Marin Grubišić
Izv. prof. dr. sc. / mag. ing. aedif.

Sveučilište J. J. Strossmayera u Osijeku (UNIOS)
Građevinski i arhitektonski fakultet Osijek (GrAFOS)
Zavod za tehničku mehaniku (ZTM)
Predsjednik Katedre za teoriju konstrukcija
Vladimira Preloga 3, **Ured II.26**, HR–31 000 Osijek, Hrvatska

marin.grubisic@gfos.hr | www.maringrubisic.com

Structural &
Earthquake Engineering